

Hybrid Liquidity Pockets

A TradingView whitepaper for fusing footprint order flow with options-chain-derived strike levels
for intraday stock, index, and commodity trading

Version 1.0 | April 11, 2026 | Prepared for the Pine v6 indicator "Hybrid Liquidity Pockets [Footprint + Options Proxy]"

Abstract

This paper specifies an intraday indicator that highlights liquidity pockets by combining two information layers: (1) executed order flow visible in TradingView’s native volume footprints and (2) strike-centered option levels derived from open interest, open-interest change, gamma, or other chain observations selected by the trader. The design is intentionally hybrid. Pine Script v6 can now request footprint data directly, but Pine does not expose an options-chain request. The current implementation therefore keeps the order-flow engine fully native while treating the options layer as a transparent, operator-supplied proxy. The result is a practical intraday map of zones where prior auction acceptance and strike-sensitive positioning may overlap [1]-[5].

Why it exists	What it uses	What it outputs
To mark price zones where accepted auction activity and option-strike context may reinforce each other.	Native footprint POC, VAH/VAL, delta, and row imbalances, plus user-supplied strikes and weights.	Ranked liquidity pockets, directional labels, dashed option-only levels, and zone entry/exit alerts.

Research note. This whitepaper defines the indicator architecture and a validation plan. It does not claim live profitability or investment suitability.

1. Problem statement and design thesis

Intraday traders routinely observe that some prices behave like magnets while others act as short-term reversal or breakout points. In practice, those levels often arise from at least two separate mechanisms. First, executed order flow can reveal where an auction found acceptance or rejection inside the bar: a point of control can persist, a value-area edge can defend, and repeated bid/ask imbalances can mark an aggressive initiative. Second, the options market can concentrate attention around discrete strikes where open interest, open-interest change, or estimated gamma sensitivity is unusually large. The indicator proposed here treats a tradable 'liquidity pocket' as a narrow price zone where those two mechanisms overlap strongly enough to matter for execution [1]-[4].

The goal is not to predict hidden order-book liquidity or reconstruct a full market-by-order model. Instead, the goal is pragmatic: provide a transparent intraday overlay that highlights price areas where recent auction acceptance and strike-centered options positioning may jointly affect how price behaves on a retest. The design therefore prioritizes four attributes: native TradingView compatibility, operator interpretability, session awareness, and adaptability across stocks, index products, and commodity futures.

Design principles

- Use the order-flow layer natively inside Pine whenever the platform exposes the necessary data.
- Keep the options layer explicit and inspectable rather than hiding it behind unverifiable heuristics.
- Represent levels as zones with decay, not as permanent horizontal lines.
- Separate probability mapping from execution judgment; the tool is a decision aid, not a complete strategy.

2. Data architecture

TradingView's Pine Script v6 now exposes true volume-footprint requests. The platform documentation and release notes state that `request.footprint()` returns a footprint object for the current bar, from which scripts can pull overall buy/sell volume, delta, Point of Control (POC), Value Area High/Low (VAH/VAL), and row-level data including buy and sell imbalances [1]-[3]. The underlying footprint is not a full depth-of-book feed; it categorizes lower-timeframe volume into 'buy' and 'sell' rows based on intrabar price action and aggregates the result into equally sized price buckets [2]. That makes it well suited for intraday auction analysis and zone construction.

The options layer uses a different path. TradingView's options chain is a platform feature that lists option contracts for stocks, indexes, ETFs, and futures and includes statistics such as strike, expiration, implied volatility, and Greeks including delta and gamma [4]. However, Pine's documented `request.*()` namespace does not include any options-chain request. TradingView also

states that it does not provide a general data API for programmatic access to indicator values or market data [2][5]. The operational implication is straightforward: a fully native options-chain reader does not exist in Pine today. The current indicator therefore takes option strikes and weights as manual or externally prepared inputs.

Figure 1. Hybrid Liquidity-Pocket Engine

Native footprint order flow is fused with options-derived strike levels to rank intraday liquidity pockets.

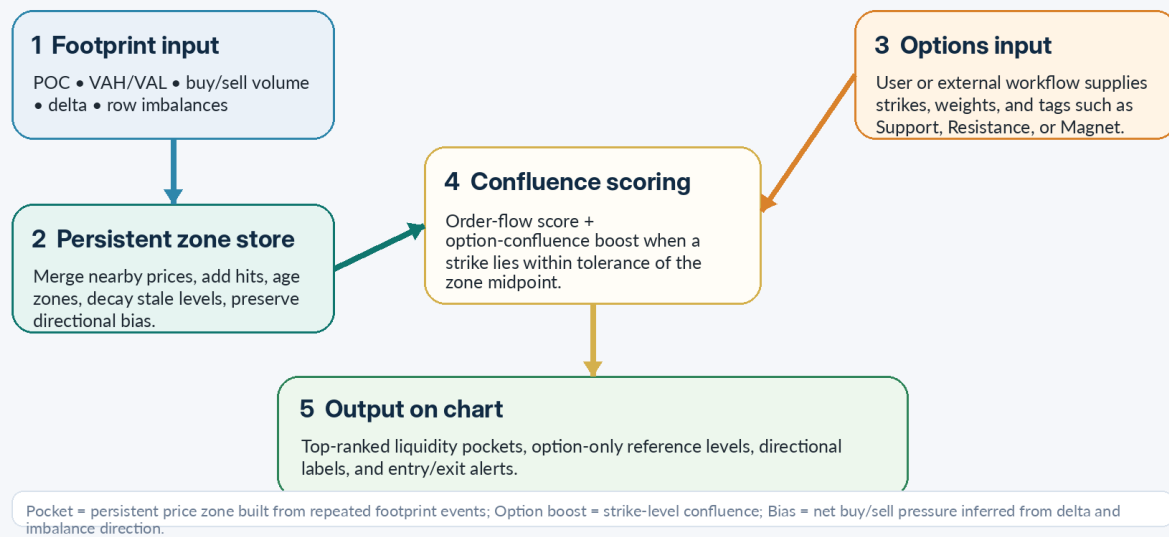


Figure 1. Native footprint data and operator-supplied options levels converge in a single pocket-ranking engine.

2.1 Native vs. proxied components

Component	Status in current TradingView build	Examples
Footprint microstructure	Native in Pine	POC, VAH/VAL, delta, total buy/sell volume, row imbalances
Zone memory and ranking	Native in Pine	Merging, decay, hit count, directional bias, top-N display
Options strikes and weights	Proxied input	Open-interest walls, estimated gamma levels, pin or magnet strikes

3. Algorithm specification

The indicator maintains a persistent collection of zones rather than treating each bar in isolation. Every bar contributes candidate prices from the footprint, and those prices are either merged into an existing zone or used to initialize a new one. Each zone stores at minimum a midpoint price, an accumulated score, a hit count, an age, and a directional bias value. The zone store is then decayed over time so that stale information fades unless new activity refreshes it. This memory is crucial

because liquidity pockets are defined by repeated structural reinforcement, not by any single footprint event.

3.1 Footprint-derived updates

On each confirmed bar, the script requests the current footprint once. If footprint data is available, the indicator adds weight to the midpoint of the POC row, to the VA boundaries, and to any rows flagged as buy or sell imbalances. TradingView defines a buy imbalance as a row whose buy volume exceeds the sell volume of the row below by the specified percentage, and a sell imbalance as a row whose sell volume exceeds the buy volume of the row above by that percentage [2]. The sign of the bar delta and the direction of imbalance rows are also accumulated into a bias variable so that zones can be labeled as buy-leaning, sell-leaning, or neutral.

Nearby prices are merged using a distance threshold measured in ticks. This prevents the script from drawing multiple nearly identical boxes when repeated order-flow events occur around the same level. The merged zone midpoint is updated as a score-weighted average, which allows heavily reinforced levels to stabilize while still drifting slightly if the market's accepted price migrates.

Working score equations

```
OFScore = decay * prior OFScore + footprint contributions
OptBoost = optBoost * nearest strike weight, if |zone midpoint - strike| <= optTol
TotalScore = OFScore + OptBoost
Pocket = [midpoint - halfWidth, midpoint + halfWidth]
where decay is the per-bar persistence factor and optTol is the strike-distance tolerance.
```

3.2 Options overlay

The options input is deliberately simple: up to several strikes are supplied with a weight and a semantic tag such as Support, Resistance, or Magnet. These inputs can be derived from the user's preferred pre-market workflow: for example, high open-interest strikes, large open-interest change, call or put walls, or externally estimated strike-level gamma exposure. Open interest is the total number of open derivative contracts carried in the market, while gamma measures the change in delta for a change in the underlying; gamma is highest near the strike when an option is at or near the money [6][7]. In the current indicator, those concepts are used heuristically through user-supplied weights rather than being estimated inside Pine.

A zone receives an options boost only when its midpoint lies within a user-defined tolerance of one of the supplied strikes. This is intentionally conservative. The indicator does not attempt to infer nonlinear hedging flows across the full surface. Instead, it asks a narrower question: does the market already care about this price in the footprint, and does that same price sit close to a strike that the trader believes is structurally important today?

3.3 Outputs and alerts

After computing composite scores, the indicator ranks all active zones and displays only the strongest pockets. Each zone is drawn as a narrow box extending forward on the chart. Color reflects dominant directional bias, and zones with options confluence can be promoted visually through a distinct border or fill treatment. Option-only levels may also be plotted as dashed guides so the trader can distinguish pure strike references from confirmed order-flow pockets. Alerts fire when price enters or exits the nearest zone.

Platform constraint

TradingView documentation makes three implementation limits explicit: volume footprints require a Premium or Ultimate plan, footprint requests can return `na` when data is unavailable, and a script may not perform more than one footprint request [2]. Because the request namespace also omits any options-chain request, the present design intentionally stops short of pretending to offer fully automated chain ingestion [2][4][5].

4. TradingView implementation for stocks and commodities

The current Pine implementation maps directly to the code structure of the indicator. The order-flow engine is responsible for requesting footprint data, updating the persistent zone store, decaying older zones, and ranking active pockets. The options engine simply stores strike, weight, and type inputs, then checks whether each active zone lies close enough to a strike to deserve a confluence boost. This separation is important because it keeps the indicator operational on TradingView today while preserving a path for later automation if the trader chooses to source option levels externally.

4.1 Core parameter groups

Group	Key inputs	Role	Practical starting point
Footprint	ticksPerRow, vaPercent, imbalancePercent	Defines footprint resolution and imbalance sensitivity	Use smaller rows for liquid instruments; keep imbalance near 300% unless testing
Pocket construction	mergeTicks, zoneHalfWidthTicks, decayFactor, maxAgeBars	Controls clustering, persistence, and display width	Start with modest merge width and a slow decay so meaningful levels persist
Order-flow weights	wPoc, wVa, wImb	Balances acceptance vs. boundary vs. aggression cues	Give POC the highest weight; keep VA lighter; use imbalances as reinforcement
Options proxy	optTolTicks, optBoost, strike, weight, type	Adds strike-level confluence without native chain access	Use only the day's most relevant 3–5 strikes; avoid overloading the chart

For equities and index products, the session filter should usually be restricted to regular trading hours so that the footprint store represents the cash-session auction that most intraday traders care about. For commodity futures, the correct choice is more product-specific. Some contracts respond best to full electronic-session memory, while others benefit from a narrower exchange-defined session that better reflects where participation is concentrated.

4.2 Suggested operating procedure

Pre-market options prep. Review the chain, identify only the highest-conviction strikes, and assign a weight that reflects your own research rather than a universal scale.

Chart setup. Choose an intraday timeframe, set the footprint row size so that important rows are visible but not noisy, and confirm the session window matches the instrument.

Intraday monitoring. Prioritize zones that are both recently reinforced and close to one of the selected strikes. A pocket that has not been refreshed for many bars should matter less.

Event discipline. Refresh or down-weight levels after major events such as earnings, inventory releases, USDA/EIA reports, macro announcements, or option-expiry transitions when the market regime may shift.

4.3 Stock vs. commodity calibration

Asset class	Chart bias	Session	Option workflow	Notes
Single-name stocks	5m–15m	RTH only	Daily strikes from stock options chain	Tighten tolerance on low-priced names; revisit after earnings or gap opens
Index ETFs / index futures proxies	3m–10m	RTH or hybrid	Use the nearest active expiries and high-gamma or pin levels	Useful on expiration-heavy days where strike clustering matters
Commodity futures	5m–15m	Exchange-specific	Use option-on-futures strikes and expiry context	Adjust tolerance for larger tick values and less granular strike ladders

5. Validation and research framework

Because this document is a specification rather than a performance report, the correct next step is empirical validation. The most useful tests compare three variants of the same workflow: footprint-only pockets, option-only levels, and the combined footprint-plus-options model. The objective is not just to ask whether a zone reacts, but whether confluence improves the quality of the reaction in a measurable way.

5.1 Event definitions

First-touch event. Price returns to a pocket after having traded away from it for a minimum separation period.

Rejection event. Price enters the pocket and then moves favorably away from it before violating a predefined adverse threshold.

Acceptance event. Price trades through the pocket, closes beyond it, and does not promptly re-enter.

Magnet event. Price accelerates toward a pocket and spends meaningful time transacting inside it before resolving.

5.2 Core evaluation metrics

Metric	How to measure it	Why it matters
First-touch response	Maximum favorable excursion before the chosen stop or invalidation threshold	Tests whether the zone is actionable on first contact
Overshoot depth	How far price travels through the pocket before reversing or accepting	Separates precise pockets from vague, noisy areas
Dwell time	Bars or seconds spent trading inside the pocket	Helps distinguish magnet behavior from sharp rejection
Relative lift	Difference between combined model and footprint-only baseline	Measures whether the options layer adds incremental signal

Validation should be stratified by instrument family, time of day, option-expiry regime, and event context. A model that behaves well in index products on expiration day may behave very differently in thin agricultural contracts or in single-name stocks during earnings season. Sensitivity analysis is also essential: if the edge vanishes when merge distance or option tolerance moves slightly, the model is probably overfit.

6. Risks, limitations, and governance

- The order-flow layer is a footprint-derived aggregation, not a direct view of hidden liquidity, iceberg detection, or full order-book depth [2].
- The options layer is operator supplied in the current Pine build, so bias can enter through strike selection and weight assignment.
- High-impact news can overwhelm both footprint memory and strike-based positioning; zones should be treated as conditional, not absolute.
- Less liquid symbols can produce unstable pockets because both footprints and options activity are sparse or noisy.
- The indicator is intended for intraday decision support and research, not as an autonomous trading system or investment recommendation.

6.1 Practical governance rules

Limit the options list. More strikes do not make the model smarter. Use only the levels that clearly dominate the current session.

Version your settings. Track which weights and tolerances belong to which product so you do not unintentionally mix configurations.

Review pocket behavior after expiry. If a dominant strike disappears after expiration, remove or refresh that level instead of letting it linger on the chart.

7. Conclusion

The Hybrid Liquidity Pockets indicator is intentionally built around what TradingView can actually do today. Pine v6 now makes footprint order flow programmatically accessible, enabling a robust auction-memory engine with POC, VA boundaries, delta, and imbalance logic [1]-[3]. By contrast, options-chain data remains outside Pine's request model, so the most honest and operational design is hybrid: native order flow plus explicit strike inputs prepared by the trader or an external workflow [2][4][5]. That architecture is not a compromise in the negative sense. It is a transparent way to combine two valuable lenses of market structure while preserving interpretability, tunability, and immediate usability for stocks and commodities.

References

- [1] TradingView, "Release notes," Pine Script documentation, January 2026 footprint update.
- [2] TradingView, "Other timeframes and data," Pine Script documentation, request.footprint section.
- [3] TradingView, "Volume footprints are now available in Pine scripts," TradingView Blog, Mar. 2, 2026.
- [4] TradingView, "Options chain overview," Help Center.
- [5] TradingView, "I need access to your API in order to get data or indicator values," Help Center.
- [6] CME Group, "Open Interest," education lesson.
- [7] CME Group, "Options Gamma – The Greeks," education lesson.

Implementation note. The accompanying Pine script uses a manual options proxy because no documented Pine request exists for the options chain. The paper and the code intentionally match.